

supervised model - module 1

① simple linear Regression



x	Data table	y
size in feet ²	price in \$1000's	
2104	400	
1416	232	
1534	315	
852	178	
...	...	
3210	870	

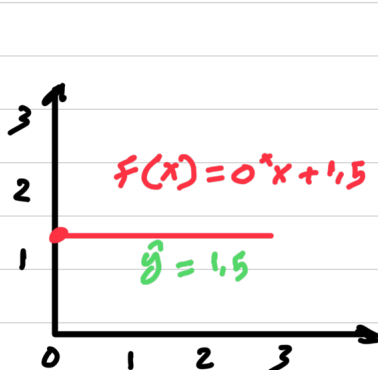
x : "input"
 y : "output"

model:

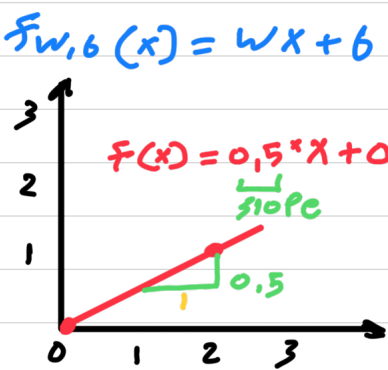
$$y = wx + b$$

=

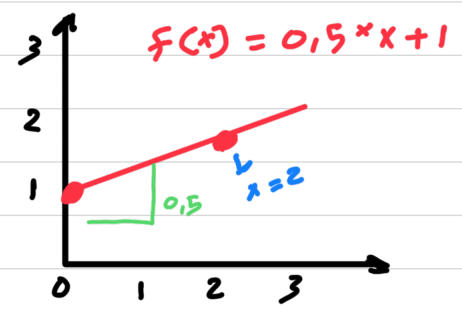
$$f_{w,b}(x) = wx + b$$



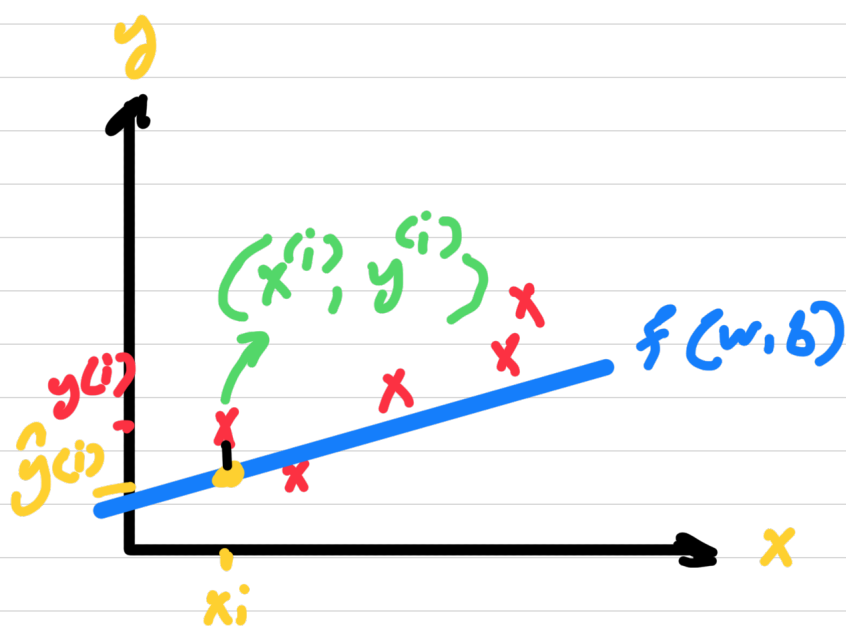
$w = 0$
 $b = 1,5$
y-intercept $\leftarrow$
 $\downarrow$
equals to $\hat{y}$



$w = 0,5$
 $b = 0$



$w = 0,5$
 $b = 1$



$$\hat{y}^{(i)} = f_{w,b}(x^{(i)})$$

$$f_{w,b}(x^{(i)}) = wx^{(i)} + b$$

Cost function:

Squared error cost function (MSE-based):

$$J(w,b) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2$$

minimum of training examples

$$J(w,b) = \frac{1}{2m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})^2$$

$\swarrow$ $\hat{y} = wx + b$

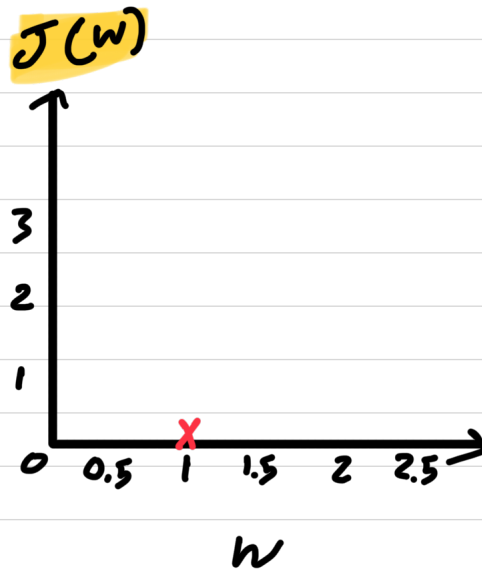
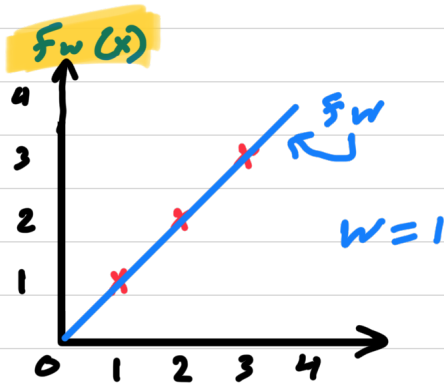
; Cost function measures the difference between the model predictions $f_{w,b}(x^{(i)})$ and the actual true values of $y^{(i)}$

• the goal is minimize the cost function $\min J(w,b)$

• simplify cost fun

$$b=0$$

$$J(w) = \frac{1}{2m} \sum_{i=1}^m (f_w(x_i) - y_i)^2$$



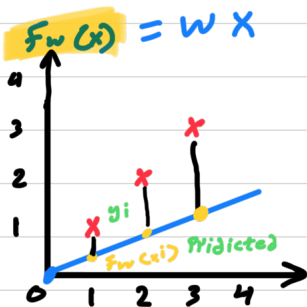
$$w=1$$

$$J(1) = 0$$

$$J(w) = \frac{1}{2m} \sum_{i=1}^m (f_w(x_i) - y_i)^2$$

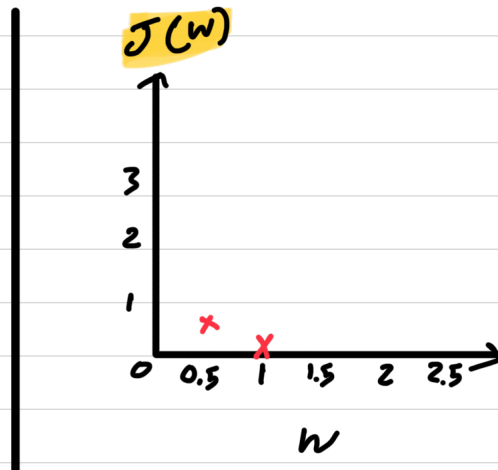
$$= \frac{1}{2m} \sum_{i=1}^m (w x_i - y_i)^2 = \frac{1}{2m} (0^2 + 0^2 + 0^2) = 0$$

$$w = 0.5$$

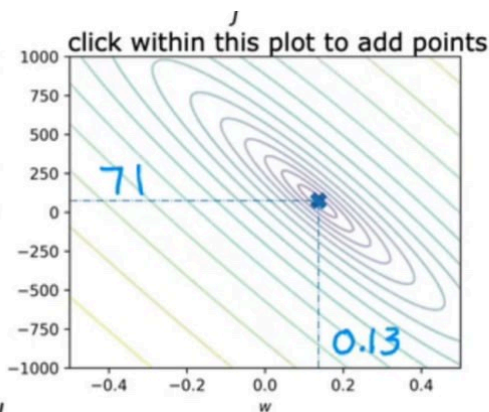
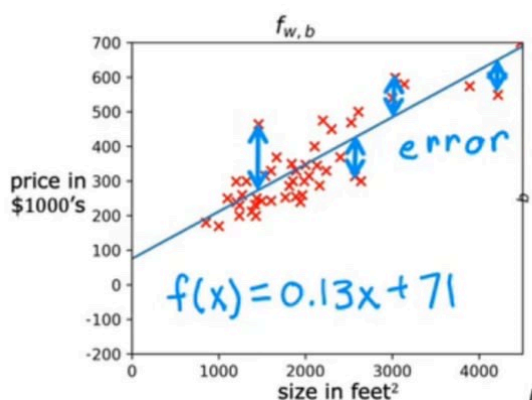


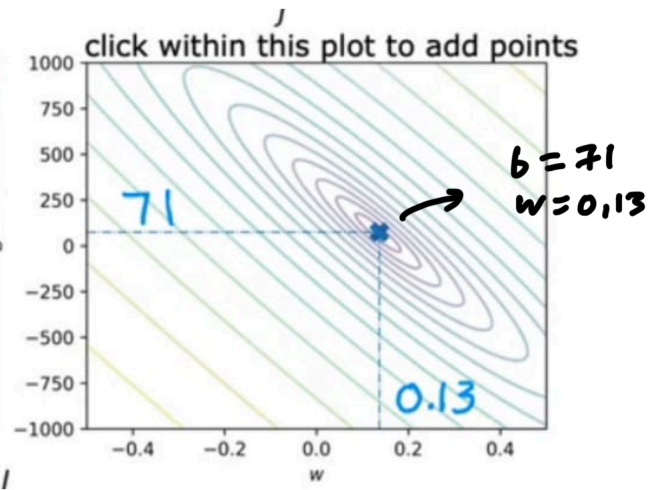
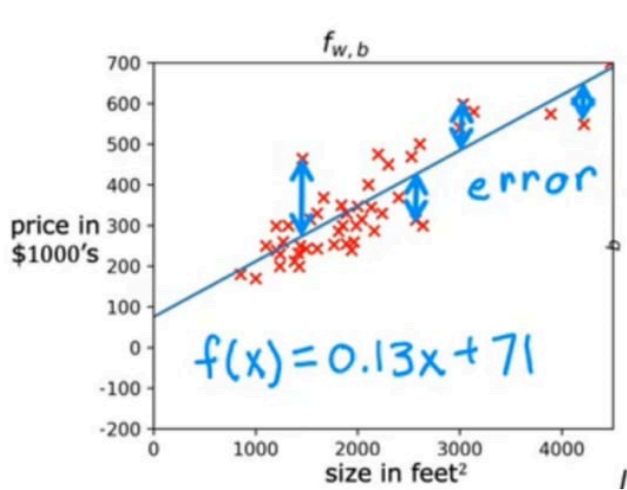
$$J(0.5) = \frac{1}{2m} ((0.5 \times 1)^2 + (1 - 2)^2 + (1.5 - 3)^2)$$

$$= \frac{1}{2(3)} [3,5] = 0.58$$



$$w = 0.5$$



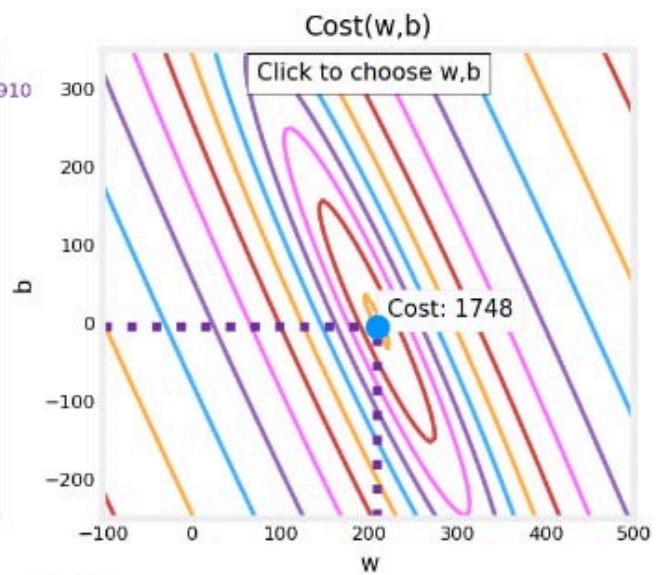
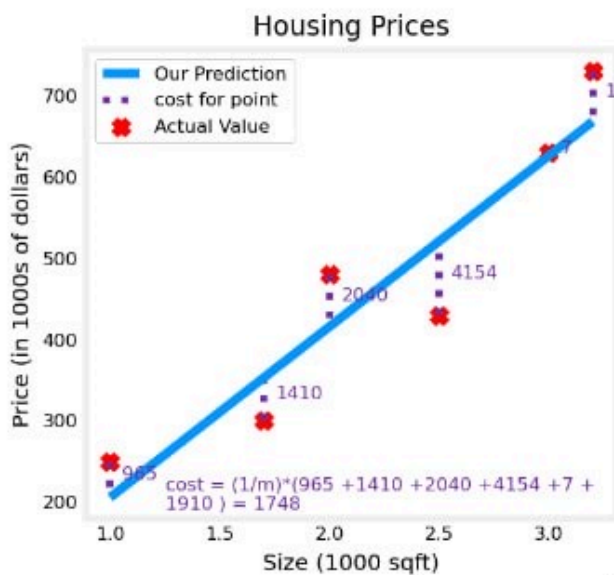


• Linear regression

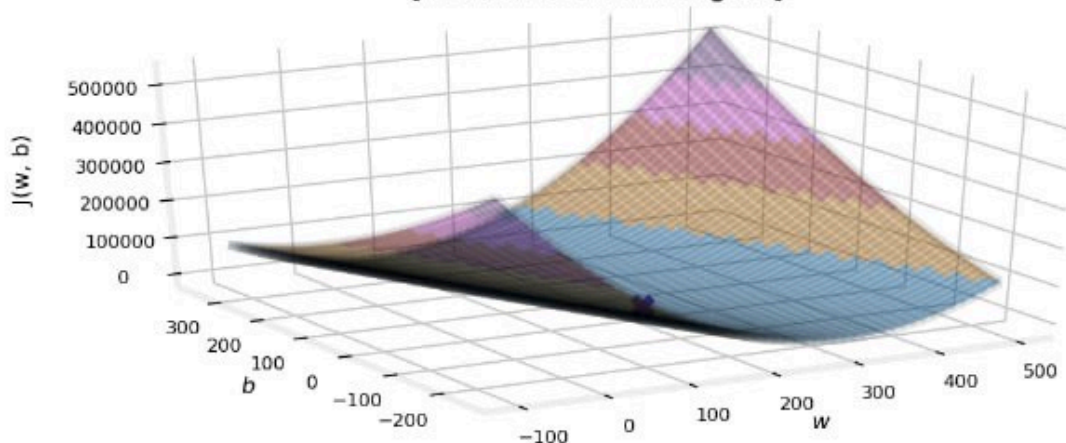
• $f_{w,b}(x) = 0.13x + 71$

• **Cost Function Counter Plot**

• the goal is to find the best param. for b, w to move the cross towards the center

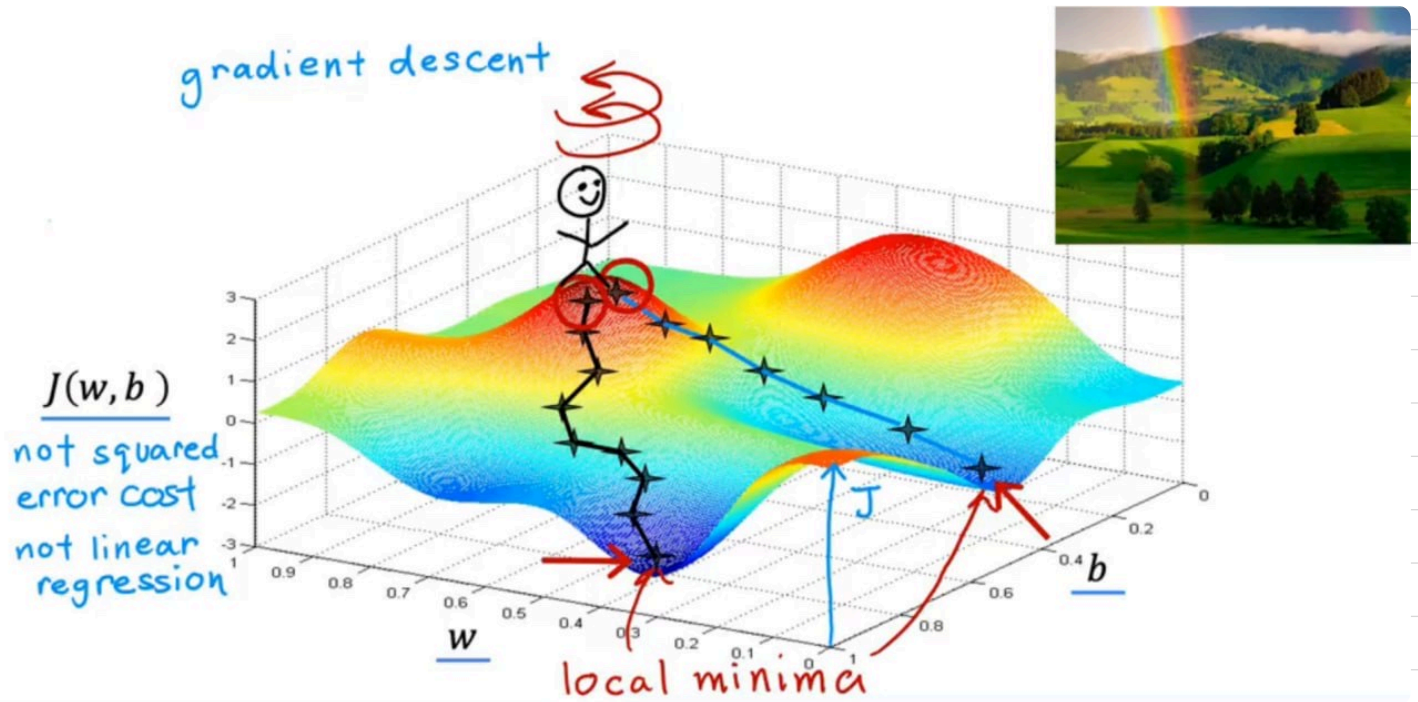


Cost(w, b)
[You can rotate this figure]



Gradient descent

- Algorithm used to minimize the cost function $J(w, b)$ by adjusting the parameters slope (w), bias (b)



Gradient descent Algorithm Implementing

each step, update w, b simultaneously

$$w = w - \alpha \frac{\partial}{\partial w} J(w, b)$$
$$b = b - \alpha \frac{\partial}{\partial b} J(w, b)$$

Derivatives

α = learning rate (step size)
 $\frac{\partial J}{\partial w} \cdot \frac{\partial J}{\partial b}$ = gradients (slope)

$$\frac{\partial J}{\partial w} = \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x_i) - y_i) \times x_i$$
$$\frac{\partial J}{\partial b} = \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x_i) - y_i)$$

↓ cost fun

$$\text{temp_w} = w - \alpha \frac{\partial}{\partial w} J(w, b)$$

$$\text{temp_b} = b - \alpha \frac{\partial}{\partial b} J(w, b)$$

$$w = \text{temp_w}$$

$$b = \text{temp_b}$$

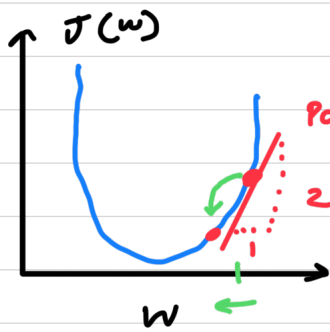
update

example

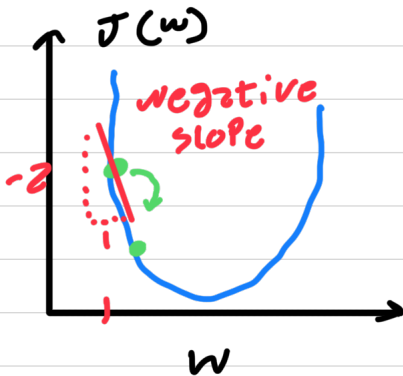
- $J(w)$

- $w = w - \alpha \frac{\partial}{\partial w} J(w)$

- $\min J(w)$



$$w = w - \alpha \times (\text{positive number})$$



$$w = w - \alpha \times (\text{negative number})$$

learning rate α

$$w = w - \alpha \frac{d}{dw} J(w)$$

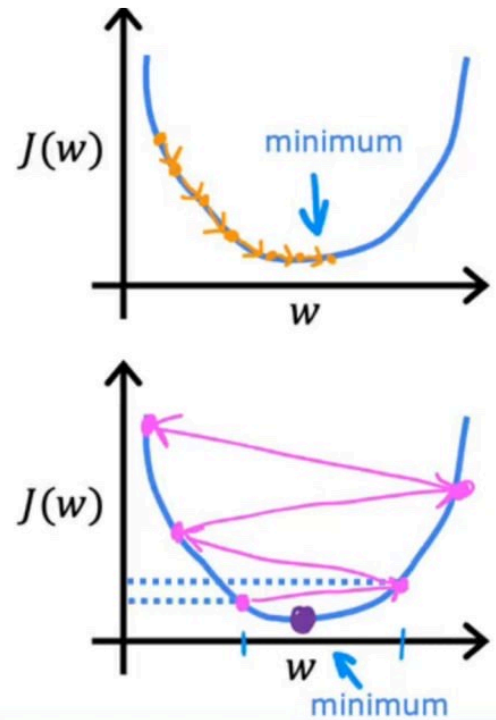
If α is too small...

Gradient descent may be slow.

If α is too large...

Gradient descent may:

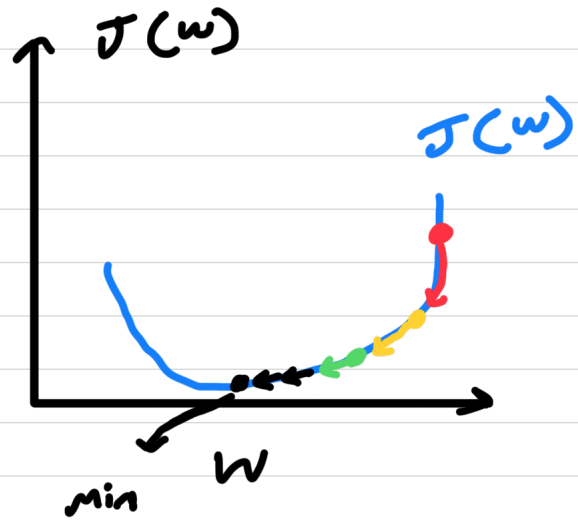
- Overshoot, never reach minimum
- Fail to converge, diverge



• It will keep the solution at local minimum

- Can reach local min with fixed learning rate η ?

$$w = w - \eta \times \frac{\partial J(w)}{\partial w}$$



- Near a local minimum gradients will take small steps • derivative become smaller
- Can reach minimum without decreasing learning rate η

price in
\$1000's

